DevOps Engineer Candidate Assessment Task

Take-home technical assessment

Overview

Build, ship, deploy and monitor a small web service end to end within **3 days (72 hours)**.

- **Target level:** DevOps Engineer (2+ years experience)
- **Time limit:** 3 days (72 hours) from receiving this task
- **Deliverable:** A public GitHub repository link containing application code, Dockerfile, docker-compose setup, CI/CD workflows, Terraform code, Kubernetes manifests, observability configuration, automation script, and a comprehensive README.md

Task Scenario

You will take a small web service from source code to a production-ready, observable, and securely deployed state. This covers containerization, a CI/CD pipeline with security gates, cloud infrastructure provisioned as code, a Kubernetes deployment, a full monitoring and logging stack, and operational automation.

We evaluate your engineering judgment and the reasoning behind your decisions as much as the working result. A smaller scope done well and explained clearly beats everything done halfway.

Step 1: Application & Containerization

1. Build a small web service (Node.js, Python FastAPI/Flask, or Go) with the following endpoints:
   - `GET /` returns `{"message": "Hello from DevOps Task!", "status": "running", "version": "<app version>"}`
   - `GET /health` is a liveness/readiness check
   - `GET /metrics` exposes Prometheus-format metrics, including request count, error count, and a request latency histogram
2. The app must read all configuration (port, log level, environment name) from environment variables, emit structured (JSON) logs to stdout, and shut down gracefully on `SIGTERM`.
3. Include unit tests for the endpoints.
4. Write a production-grade Dockerfile that:
   - Uses a multi-stage build with a minimal final image (distroless or Alpine). State your final image size in the README.
   - Pins the base image to a specific version or digest.
   - Runs as a non-root user.
   - Includes a `HEALTHCHECK`, OCI labels, and a proper `.dockerignore`.
5. Write a `docker-compose.yml` that runs the app locally on port 8080, with a health check, restart policy, and resource limits defined.

Step 2: CI/CD Pipeline (GitHub Actions)

Create workflows under `.github/workflows/` that implement the following.

On every Pull Request:

1. Code linting, unit tests, and test coverage reporting.

2. Dockerfile linting (e.g. `hadolint`) and shell script linting (`shellcheck`).
3. Build the image and run a vulnerability scan (e.g. Trivy). The pipeline must fail on CRITICAL vulnerabilities.

On push to `main`:

1. Build a multi-architecture image (`linux/amd64` and `linux/arm64`) with layer caching enabled.
2. Tag images with the git commit SHA and a semantic version. Do not rely on `latest` alone.
3. Push to a container registry (Docker Hub, GHCR, or Amazon ECR). For AWS, prefer GitHub OIDC over long-lived access keys.
4. Deploy the new image to the EC2 instance provisioned in Step 3 (via SSM or SSH), run a smoke test against `/health`, and automatically roll back to the previous image tag if it fails.

No secrets may be hardcoded. Document which secrets and variables are required and how they are configured. If you have destroyed your AWS resources after testing, evidence of a successful deploy run (workflow logs or screenshots) is sufficient.

Step 3: Infrastructure as Code (Terraform)

Provision a simple deployment target on AWS using Terraform:

1. An EC2 instance in a VPC (the default VPC is acceptable) with a security group that exposes only the required ports.
2. An IAM instance profile scoped to the minimum permissions needed (e.g. pull from ECR, write to CloudWatch Logs). No static access keys anywhere.
3. User data or a bootstrap script that installs Docker and runs the application container.
4. Container logs shipped to CloudWatch Logs with a defined retention period.
5. Remote state stored in S3 with state locking enabled.
6. Variables so the same code can provision both `staging` and `production` environments.

The AWS Free Tier is sufficient. A clean `terraform plan` output plus evidence of a successful `apply` (screenshot or logs) is acceptable. Please destroy resources after testing.

Step 4: Kubernetes Deployment

Deploy the application to a local Kubernetes cluster (kind, minikube, or k3d). A managed cluster (EKS/GKE/AKS) is **not** required.

1. Write Kubernetes manifests (plain YAML or Kustomize) under a `k8s/` directory that include:
   - A `Deployment` with at least 2 replicas, a rolling update strategy, and liveness and readiness probes using `/health`.
   - CPU and memory requests and limits.
   - A `Service` and an `Ingress` (or a documented `port-forward` alternative).
   - A `ConfigMap` for configuration and a `Secret` for sensitive values (no secrets committed in plain text; document your approach).
   - A `securityContext` that runs the container as non-root with a read-only root filesystem.
2. Add a `HorizontalPodAutoscaler` based on CPU usage.
3. Demonstrate a zero-downtime rolling update to a new image version, and a rollback using `kubectl rollout undo`. Include the commands and output in the README.
4. In the README, briefly explain how this setup would change for production on EKS (e.g. ingress controller, secrets management, node autoscaling, IAM roles for service accounts).

Step 5: Observability Stack

Extend the local setup with a full monitoring and logging stack using **Prometheus, Grafana, Loki, Grafana Alloy, Node Exporter, and cAdvisor**. All six components are mandatory. You may add them to the main `docker-compose.yml` (e.g. using Compose profiles) or provide a separate `docker-compose.monitoring.yml`.

1. **Metrics collection:**
   - Prometheus scrapes the app's `/metrics` endpoint, Node Exporter (host metrics), and cAdvisor (container metrics).
   - Set an explicit retention period for Prometheus data.
2. **Log collection:**
   - Grafana Alloy collects logs from all containers (via Docker service discovery) and ships them to Loki.
   - Parse the app's structured JSON logs so that fields such as log level and request path can be used as labels or filters in Grafana. Keep label cardinality low and explain your choices.
   - Set an explicit retention period for Loki data.
3. **Visualization (Grafana):**
   - Data sources (Prometheus and Loki) and dashboards must be **provisioned as code** from files in the repository, not configured manually through the UI.
   - Provide at least three dashboards, exported as JSON in the repository:
     - **Application:** request rate, error rate, and latency (p50/p95/p99).
     - **Host:** CPU, memory, disk, and network usage from Node Exporter.
     - **Containers:** per-container CPU and memory usage from cAdvisor, including memory usage relative to the configured limit.
   - At least one dashboard must include a Loki logs panel next to the related metrics, so an error spike can be investigated in the same view.
4. **Alerting:**
   - Define at least four alerting rules (Prometheus rules or Grafana alerting), for example: app instance down, high error rate, host disk usage above 80%, and container memory close to its limit.
   - Explain the reasoning behind each threshold and the `for` duration you chose.
5. **Hygiene and security:**
   - Grafana admin credentials must come from environment variables or a secret, not be hardcoded.
   - Only Grafana is exposed to the host. Prometheus, Loki, Alloy, Node Exporter, and cAdvisor communicate over an internal Docker network.
   - Pin all image versions.
   - cAdvisor and Node Exporter need access to host resources. Document which mounts and privileges you granted and why.
6. In the README, include a screenshot of each dashboard and a short walkthrough answering: "*An error spike appears on the Application dashboard. How do you find the cause using these tools?*"

Step 6: Linux Automation Scripting

Write a Bash script `system_health.sh` suitable for running on production servers:

1. Collect CPU usage, memory usage, disk usage (all mounted filesystems), load average, and the top 5 processes by memory.
2. Append a timestamped entry to `health_log.txt`, with an option to output JSON (`--json`).

3. Accept configurable thresholds via flags (e.g. `--disk 80 --cpu 90 --mem 85`) with sensible defaults, and print `[WARNING]` messages when any threshold is exceeded.
4. Return meaningful exit codes (0 = healthy, 1 = warning, 2 = critical) so it can be used by monitoring tools.
5. Implement basic log rotation so the log file does not grow indefinitely.
6. Use `set -euo pipefail`, pass `shellcheck` with no warnings, and include a `--help` option.
7. Provide a cron entry or a systemd service + timer to run it every 5 minutes.

Step 7: Documentation (README.md)

Your README must let a reviewer run everything without guessing. Include:

1. Prerequisites and step-by-step instructions to run the app and the observability stack locally with `docker compose up`, including how to access Grafana and the dashboards.
2. Instructions for running the tests, the health script, the Terraform code, and the Kubernetes deployment.
3. An architecture diagram covering Code Commit → CI (lint, test, scan) → Registry → Deployment to EC2, along with the Kubernetes deployment flow and the observability stack (how metrics and logs flow from the app and host into Grafana).
4. A **Design Decisions** section explaining key trade-offs (e.g. distroless vs Alpine, EC2 vs ECS vs EKS, your tagging and secrets strategy, log label choices).
5. A **Deployment & Rollback Strategy** section: how a release reaches production safely, and how you would roll back.
6. A **Security Considerations** section covering secrets management, image scanning, IAM, and network exposure.
7. A rough monthly AWS cost estimate for running this in production.
8. A **What I would improve with more time** section.
9. A short note on whether and how you used AI tools during this task. Using them is allowed; we only ask that you can explain every part of your submission.

Bonus (Optional)

- Alertmanager with alert routing to Slack or Discord.
- Deploy the monitoring stack on the Kubernetes cluster (e.g. using the kube-prometheus-stack Helm chart).
- Distributed tracing with OpenTelemetry and Grafana Tempo.
- Package the Kubernetes manifests as a Helm chart with separate values files for `staging` and `production`.
- Send alerts from `system_health.sh` to a Slack/Discord webhook (URL read from an environment variable).
- Image signing (e.g. Cosign) and SBOM generation in the pipeline.
- Ansible playbook for basic server hardening and deploying the health script.

Evaluation Criteria

- **Containerization:** image size, layer efficiency, security hardening, and runtime hygiene.
- **CI/CD Design:** pipeline structure, security gates, caching, tagging strategy, and safe deployment with rollback.
- **Infrastructure as Code:** least privilege, reusability, and state management.

- **Kubernetes:** correct use of probes, resource management, security context, and safe rollout/rollback.
- **Observability:** meaningful metrics and dashboards, structured log handling, actionable alerts with justified thresholds, and the stack provisioned as code.
- **Scripting & Automation:** robustness, error handling, configurability, and readability.
- **Git Practices:** clean, atomic commits with clear messages, and use of branches/PRs.
- **Documentation & Judgment:** clarity, reproducibility, and the quality of the reasoning behind design decisions.